

5.11

Of these two types of programs:

- a. I/O-bound
- b. CPU-bound

which is more likely to have voluntary context switches, and which is more likely to have nonvoluntary context switches? Explain your answer.

For I/O-bound we require voluntary context switches.  

For CPU-bound we require non-voluntary context switches.

A voluntary context switches occurs when process has given up control of the CPU because it requires a resource that is currently unavailable(such as blocking for I/O). It happen frequently in normal system operation. Voluntary context switching initiated with a call to the `sleep()` routine.

A non-voluntary context switches occurs when the CPU has taken away from a process, such as when it's time slice has expired or it has been preempted by a high-priority process. It is forced by direct invocation of the low-level context-switching mechanism embodied in the `mi_switch()` and `setrunnable()` routines

5.12

5.2 Discuss how the following pairs of scheduling criteria conflict in certain settings.

- a. CPU utilization and response time
- b. Average turnaround time and maximum waiting time
- c. I/O device utilization and CPU utilization

Answer:

- a. CPU utilization and response time: CPU utilization is increased if the overheads associated with context switching is minimized. The context switching overheads could be lowered by performing context switches infrequently. This could, however, result in increasing the response time for processes.
- b. Average turnaround time and maximum waiting time: Average turnaround time is minimized by executing the shortest tasks first. Such a scheduling policy could, however, starve long-running tasks and thereby increase their waiting time.
- c. I/O device utilization and CPU utilization: CPU utilization is maximized by running long-running CPU-bound tasks without performing context switches. I/O device utilization is maximized by scheduling I/O-bound jobs as soon as they become ready to run, thereby incurring the overheads of context switches.

5.13

5.3 One technique for implementing **lottery scheduling** works by assigning processes lottery tickets, which are used for allocating CPU time. Whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the CPU. The BTV operating system implements lottery

scheduling by holding a lottery 50 times each second, with each lottery winner getting 20 milliseconds of CPU time ($20 \text{ milliseconds} \times 50 = 1 \text{ second}$). Describe how the BTV scheduler can ensure that higher-priority threads receive more attention from the CPU than lower-priority threads.

Answer:

By assigning more lottery tickets to higher-priority processes.

5.14

5.4 In this chapter, we discussed possible race conditions on various kernel data structures. Most scheduling algorithms maintain a **run queue**, which lists processes eligible to run on a processor. On multicore systems, there are two general options: (1) each processing core has its own run queue, or (2) a single run queue is shared by all processing cores. What are the advantages and disadvantages of each of these approaches?

Answer:

The primary advantage of each processing core having its own run queue is that there is no contention over a single run queue when the scheduler is running concurrently on 2 or more processors. When a scheduling decision must be made for a processing core, the scheduler only need to look no further than its private run queue. A disadvantage of a single run queue is that it must be protected with locks to prevent a race condition and a processing core may be available to run a thread, yet it must first acquire the lock to retrieve the thread from the single queue. However, load balancing would likely not be an issue with a single run queue, whereas when each processing core has its own run queue, there must be some sort of load balancing between the different run queues.

5.15

5.5 Consider the exponential average formula used to predict the length of the next CPU burst. What are the implications of assigning the following values to the parameters used by the algorithm?

- a. $\alpha = 0$ and $\tau_0 = 100$ milliseconds
- b. $\alpha = 0.99$ and $\tau_0 = 10$ milliseconds

Answer:

When $\alpha = 0$ and $\tau_0 = 100$ milliseconds, the formula always makes a prediction of 100 milliseconds for the next CPU burst. When $\alpha = 0.99$ and $\tau_0 = 10$ milliseconds, the most recent behavior of the process is given much higher weight than the past history associated with the process. Consequently, the scheduling algorithm is almost memoryless, and simply predicts the length of the previous burst for the next quantum of CPU execution.

5.16

5.6 A variation of the round-robin scheduler is the **regressive round-robin scheduler**. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process (CPU-bound or I/O-bound) does the regressive round-robin scheduler favor? Explain.

Answer:

This scheduler would favor CPU-bound processes as they are rewarded with a longer time quantum as well as priority boost whenever they consume an entire time quantum. This scheduler does not penalize I/O-bound processes as they are likely to block for I/O before consuming their entire time quantum, but their priority remains the same.

5.17

4. 5.17. Consider the following set of processes, with the length of the CPU-burst time given in milliseconds:

Process	Burst Time	Priority
P_1	5	4
P_2	3	1
P_3	1	2
P_4	7	2
P_5	4	3

The processes are assumed to have arrived in the order P_1, P_2, P_3, P_4, P_5 , all at time 0.

- Draw four Gantt charts illustrating the execution of these processes using FCFS, SJF, a non-preemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2) scheduling.
- What is the turnaround time of each process for each of the scheduling algorithms in part a?
- What is the waiting time of each process for each of the scheduling algorithms in part a?
- Which of the schedules in part a results in the minimal average waiting time (over all processes)?

Answer: a. The four Gantt charts not shown.

b. Turnaround time

	FCFS	SJF	Priority	RR
P_1	5	13	5	17
P_2	8	4	20	12
P_3	9	1	10	5
P_4	16	20	17	20
P_5	20	8	9	16

c. Waiting time (turnaround time minus burst time)

	FCFS	SJF	Priority	RR
P_1	0	8	0	12
P_2	5	1	17	9
P_3	8	0	9	4
P_4	9	13	10	14
P_5	16	4	5	12

d. Shortest Job First (SJF) has the minimum average waiting time.

5.18

5.18) The following processes are being scheduled using a preemptive, priority-based, round-robin scheduling algorithm: Each process is assigned a numerical priority, with a higher number indicating a higher relative priority. The scheduler will execute the highest-priority process. For processes with the same priority, a round-robin scheduler will be used with a time quantum of 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

- Show the scheduling order of the processes using a Gantt chart.
- What is the turnaround time for each process?
- What is the waiting time for each process?

Process	Priority	Burst	Arrival
-----	-----	-----	-----
P1	8	15	0
P2	3	20	0
P3	4	20	20
P4	4	20	25
P5	5	5	45
P6	5	15	55

Answer: a. time=0 P1
 time=15 P2
 time=20 P3
 time=30 P4
 time=40 P3
 time=45 P5
 time=50 P3
 time=55 P6
 time=70 P4
 time=80 P2

b. P1 = 15, P2 = 95, P3 = 35, P4 = 55, P5 = 5, P6 = 15

c. P1 = 0, P2 = 75, P3 = 15, P4 = 35, P5 = 0, P6 = 0

5.19

5.9 The `nice` command is used to set the nice value of a process on Linux, as well as on other UNIX systems. Explain why some systems may allow any user to assign a process a nice value ≥ 0 yet allow only the root user to assign nice values < 0 .

Answer:

Nice values < 0 are assigned a higher relative priority and such systems may not allow non-root processes to assign themselves higher priorities.

5.20

5.10 Which of the following scheduling algorithms could result in starvation?

- a. First-come, first-served
- b. Shortest job first
- c. Round robin
- d. Priority

Answer:

Shortest job first and priority-based scheduling algorithms could result in starvation.

5.21

5.11 Consider a variant of the RR scheduling algorithm where the entries in the ready queue are pointers to the PCBs.

- a. What would be the effect of putting two pointers to the same process in the ready queue?
- b. What would be two major advantages and disadvantages of this scheme?
- c. How would you modify the basic RR algorithm to achieve the same effect without the duplicate pointers?

Answer:

- a. In effect, that process will have increased its priority since by getting time more often it is receiving preferential treatment.
- b. The advantage is that more important jobs could be given more time, in other words, higher priority in treatment. The consequence, of course, is that shorter jobs will suffer.
- c. Allot a longer amount of time to processes deserving higher priority. In other words, have two or more quanta possible in the Round-Robin scheme.

5.22

5.12 Consider a system running ten I/O-bound tasks and one CPU-bound task. Assume that the I/O-bound tasks issue an I/O operation once for every millisecond of CPU computing and that each I/O operation takes 10 milliseconds to complete. Also assume that the context-switching overhead is 0.1 millisecond and that all processes are long-running tasks. Describe the CPU utilization for a round-robin scheduler when:

- a. The time quantum is 1 millisecond
- b. The time quantum is 10 milliseconds

Answer:

- a. The time quantum is 1 millisecond: Irrespective of which process is scheduled, the scheduler incurs a 0.1 millisecond context-switching cost for every context-switch. This results in a CPU utilization of $1/1.1 * 100 = 91\%$.
- b. The time quantum is 10 milliseconds: The I/O-bound tasks incur a context switch after using up only 1 millisecond of the time quantum. The time required to cycle through all the processes is therefore $10 * 1.1 + 10.1$ (as each I/O-bound task executes for 1 millisecond and then incur the context switch task, whereas the CPU-bound task executes for 10 milliseconds before incurring a context switch). The CPU utilization is therefore $20/21.1 * 100 = 94\%$.

5.23

5.13 Consider a system implementing multilevel queue scheduling. What strategy can a computer user employ to maximize the amount of CPU time allocated to the user's process?

Answer:

The program could maximize the CPU time allocated to it by not fully utilizing its time quantum. It could use a large fraction of its assigned quantum, but relinquish the CPU before the end of the quantum, thereby increasing the priority associated with the process.

5.24

5.14 Consider a preemptive priority scheduling algorithm based on dynamically changing priorities. Larger priority numbers imply higher priority. When a process is waiting for the CPU (in the ready queue, but not running), its priority changes at a rate α ; when it is running, its priority changes at a rate β . All processes are given a priority of 0 when they enter the ready queue. The parameters α and β can be set to give many different scheduling algorithms.

- a. What is the algorithm that results from $\beta > \alpha > 0$?
- b. What is the algorithm that results from $\alpha < \beta < 0$?

Answer:

- a. FCFS
- b. LIFO

5.25

5.15 Explain the differences in how much the following scheduling algorithms discriminate in favor of short processes:

- a. FCFS
- b. RR
- c. Multilevel feedback queues

Answer:

- a. FCFS—discriminates against short jobs since any short jobs arriving after long jobs will have a longer waiting time.
- b. RR—treats all jobs equally (giving them equal bursts of CPU time) so short jobs will be able to leave the system faster since they will finish first.
- c. Multilevel feedback queues work similar to the RR algorithm—they discriminate favorably toward short jobs.

5.26

5.26 Describe why a shared ready queue might suffer from performance problems in an SMP environment.

ANS

SMP is basically Symmetric multiprocessing in which multiple processes are running at a same time which often causes inefficiency the reason why shared ready queue will suffer in performance because the SMP causes data loss and shared queue shares its resources with SMP causing inefficiency and will cause more time and resources so that's why we don't use shared ready queue for SMP environment because it creates lot of performance related issue and risk of data loss is more than in normal occassion

5.27

5.27) Consider a load-balancing algorithm that ensures that each queue has approximately the same number of threads, independent of priority. How effectively would a priority-based scheduling algorithm handle this situation if one run queue had all high-priority threads and a second queue had all low-priority threads?

Answer: In this situation, the low-priority threads would get approximately the same amount of CPU time as the high-priority threads. This would not accurately reflect a priority-based scheduling algorithm, which is supposed to favor high-priority threads.

5.28

- 5.28 Assume that an SMP system has private, per-processor run queues. When a new process is created, it can be placed in either the same queue as the parent process or a separate queue.
- What are the benefits of placing the new process in the same queue as its parent?
 - What are the benefits of placing the new process in a different queue?

ANS

Answer :-

(i). If the new process is placed in the same queue as its parent , then it is just because of **some similar strategy which is being followed by all the process in that queue** . Although there will be queue for the new process , but there will be a surity that all the processes will be processed smoothly without getting hanged at any point until the system throws some error . So having similar type os strategy , it is beneficial for the new process to be with its parent process .

(ii). If the new process is being scheduled in a different queue , it will be just for the shake of being **reducing the time that will be taken by the process** . There is some small chances that the process will be processed slowly i.e execution time can be slow but the *compilation time or the taken taken by a process* in the queue of its parent process will be much more as compared to that of the process placed in different queue . So for reducing time , the system might schedule the new process in a difefrent queue .

5.29

- 5.29 Assume that a thread has blocked for network I/O and is eligible to run again. Describe why a NUMA-aware scheduling algorithm should reschedule the thread on the same CPU on which it previously ran.

ANS

Answer :- NUMA -aware scheduling algorithm should reschedule The thread on the same cpu on which it previously ran because it supports the spot of processes to memory and by dispatching the process on the very same node and then it can be done to process the dispatching processes on to the same processor. And also the thread in I/O spent a significant part of the time of execution and also waits for the data from the storage to citation. And also the thread are those that the os is not aware of. But when the the I/o process finally completes now it will not execute for a very long time before they start the next i/o operation.

But when the cpu thread are blocked then they cannot be run for a long time and then that is why the NUMA scheduling should reschedule the thread on the same cpu.

5.30

5.16 Using the Windows scheduling algorithm, determine the numeric priority of each of the following threads

- a. A thread in the `REALTIME_PRIORITY_CLASS` with a relative priority of `HIGHEST`.

- b. A thread in the `NORMAL_PRIORITY_CLASS` with a relative priority of `NORMAL`.
c. A thread in the `HIGH_PRIORITY_CLASS` with a relative priority of `ABOVE_NORMAL`.

Answer:

- a. 26
b. 8
c. 14

5.31

5.17 Assuming that no threads belong to the `REALTIME_PRIORITY_CLASS` and that none may be assigned a `TIME_CRITICAL` priority, what combination of priority class and priority corresponds to the highest possible relative priority in Windows scheduling?

Answer:

HIGH priority class and HIGHEST priority within that class. (numeric priority of 15)

5.32

5.32 Consider the scheduling algorithm in the Solaris operating system for time-sharing threads.

- a. What is the time quantum (in milliseconds) for a thread with priority 15? With priority 40?
- b. Assume that a thread with priority 50 has used its entire time quantum without blocking. What new priority will the scheduler assign this thread?
- c. Assume that a thread with priority 20 blocks for I/O before its time quantum has expired. What new priority will the scheduler assign this thread?

a. Priority 15 will have time quantum of 160ms and the priority 40 will have quantum of 40ms.

b. The new priority which is a lower priority will be 40.

c. The new priority(which is a higher priority will be 52.

5.33

5.19 Assume that two tasks A and B are running on a Linux system. The nice values of A and B are -5 and $+5$, respectively. Using the CFS scheduler as a guide, describe how the respective values of `vruntime` vary between the two processes given each of the following scenarios:

- Both A and B are CPU-bound.
- A is I/O-bound, and B is CPU-bound.
- A is CPU-bound, and B is I/O-bound.

Answer:

- Since A has a higher priority than B , `vruntime` will move more slowly for A than B . If both A and B are CPU-bound (that is they both use the CPU for as long as it is allocated to them), `vruntime` will generally be smaller for A than B , and hence A will have a greater priority to run over B .
- In this situation, `vruntime` will be much smaller for A than B as (1) `vruntime` will move more slowly for A than B due to priority differences, and (2) A will require less CPU-time as it is I/O-bound.
- This situation is not as clear, and it is possible that B may end up running in favor of A as it will be using the processor less than A and in fact its value of `vruntime` may in fact be less than the value of `vruntime` for B .

5.34

5.21 Under what circumstances is rate-monotonic scheduling inferior to earliest-deadline-first scheduling in meeting the deadlines associated with processes?

Answer:

Consider two processes P_1 and P_2 where $p_1 = 50$, $t_1 = 25$ and $p_2 = 75$, $t_2 = 30$. If P_1 were assigned a higher priority than P_2 , then the following scheduling events happen under rate-monotonic scheduling. P_1 is scheduled at $t = 0$, P_2 is scheduled at $t = 25$, P_1 is scheduled at $t = 50$, and P_2 is scheduled at $t = 75$. P_2 is not scheduled early enough to meet its deadline. The earliest deadline schedule performs the following scheduling events: P_1 is scheduled at $t = 0$, P_2 is scheduled at $t = 25$, P_1 is scheduled at $t = 55$, and so on. This schedule actually meets the deadlines and therefore earliest-deadline-first scheduling is more effective than the rate-monotonic scheduler.

5.35

5.22 Consider two processes, P_1 and P_2 , where $p_1 = 50$, $t_1 = 25$, $p_2 = 75$, and $t_2 = 30$.

- Can these two processes be scheduled using rate-monotonic scheduling? Illustrate your answer using a Gantt chart such as the ones in Figure 5.16–Figure 5.19.
- Illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling.

Answer:

Consider when P_1 is assigned a higher priority than P_2 with the rate monotonic scheduler. P_1 is scheduled at $t = 0$, P_2 is scheduled at $t = 25$, P_1 is scheduled at $t = 50$, and P_2 is scheduled at $t = 75$. P_2 is not scheduled early enough to meet its deadline. When P_1 is assigned a lower priority than P_2 , then P_1 does not meet its deadline since it will not be scheduled in time.

5.36

5.23 Explain why interrupt and dispatch latency times must be bounded in a hard real-time system.

Answer:

following tasks: save the currently executing instruction, determine the type of interrupt, save the current process state, and then invoke the appropriate interrupt service routine. Dispatch latency is the cost associated with stopping one process and starting another. Both interrupt and dispatch latency needs to be minimized in order to ensure that real-time tasks receive immediate attention. Furthermore, sometimes interrupts are disabled when kernel data structures are being modified, so the interrupt does not get serviced immediately. For hard real-time systems, the time-period for which interrupts are disabled must be bounded in order to guarantee the desired quality of service.

5.37

5.37 Describe the advantages of using heterogeneous multiprocessing in a mobile system.

ANS

2. The advantage of heterogenous multiprocessing is every process have equal probability to get cpu for execution , i,e there is no starvation and no priority to any process .